SRE Interview Questions

Introductory Questions

1. Tell us about yourself.

Answer: Looks for what things the candidate emphasizes about their background, skills, and experiences. A good response highlights relevant experience in DevOps, cloud infrastructure, automation, and reliability engineering. It should also touch on problem-solving abilities, collaboration with teams, and a passion for building scalable systems.

---

2. Why are you interested in this SRE role?

Answer: Look for enthusiasm about the SRE field, understanding of the role's responsibilities, and alignment with the company's mission. A strong answer might include: * Passion for improving system reliability and performance. * Interest in automation and infrastructure as code. * Desire to work in a collaborative environment with developers and operations teams. * Excitement about the company's products and how they can contribute to its success. * Alignment with personal career goals in site reliability engineering. * Eagerness to learn and grow in a challenging technical environment. * Commitment to maintaining high availability and user satisfaction. * Experience with relevant technologies like Kubernetes, Terraform, and cloud platforms. * Understanding of the importance of monitoring, incident response, and continuous improvement in SRE practices.

this list is not exhaustive but provides a comprehensive view of what a good answer might include.

---

3. What are your strengths and how do they help in this role?

Answer: Look for strengths that align with SRE responsibilities, such as: * Strong problem-solving skills to diagnose and fix issues quickly. * Proficiency in automation tools (e.g., Terraform, Jenkins) to streamline operations. * Experience with cloud platforms (e.g., Azure, GCP) for scalable infrastructure management. * Collaboration skills to work effectively with cross-functional teams. * Attention to detail for monitoring and maintaining system health. * Ability to handle high-pressure situations during incidents. * Strong communication skills for clear documentation and incident reporting.

This list is not exhaustive but provides a comprehensive view of what a good answer might include.

---

4. Can you describe a challenge you faced and how you overcame it?

Answer: Look for a specific example that demonstrates problem-solving, resilience, and learning. A strong response might include: * Situation: Briefly describe the challenge (e.g., a major outage, deployment failure). * Task: Explain your role and what needed to be accomplished. * Action: Detail the steps you took to address the issue (e.g., troubleshooting, collaboration, implementing a fix). * Result: Share the outcome (e.g., restored service, improved processes to prevent future issues). * Reflection: Mention what you learned from the experience and how it improved your skills or approach. * Impact: Highlight any positive impact on the team or organization, such as reduced downtime or improved reliability.

This list is not exhaustive but provides a comprehensive view of what a good answer might include.

---

5. How do you stay updated with the latest technologies and best practices?

Answer: Look for a proactive approach to continuous learning. A strong answer might include: * Following industry blogs, forums, and news sites (e.g., Hacker News, DevOps.com). * Participating in online communities (e.g., Stack Overflow, Reddit). * Attending webinars, conferences, and meetups related to SRE and DevOps. * Taking online courses and certifications (e.g., Coursera, Udemy, AWS certifications). * Experimenting with new tools and technologies in personal projects or labs. * Reading books and whitepapers on relevant topics. * Engaging with peers and mentors to share knowledge and experiences. * Subscribing to newsletters and podcasts focused on site reliability engineering and cloud computing.

This list is not exhaustive but provides a comprehensive view of what a good answer might include.

---

6. How do you prioritize tasks when you have multiple issues to resolve?

Answer: Look for a structured approach to prioritization. A strong answer might include: * Assessing the impact and urgency of each issue (e.g., customer-facing outages vs. internal bugs). * Using frameworks like the Eisenhower Matrix (urgent vs. important) to categorize tasks. * Communicating with stakeholders to understand business priorities. * Breaking down larger tasks into smaller, manageable steps. * Allocating time for high-priority tasks while scheduling lower-priority ones. * Using tools like Jira or Trello to track and manage tasks. * Regularly reviewing and adjusting priorities based on new information or changes in the environment.

This list is not exhaustive but provides a comprehensive view of what a good answer might include.

---

7. What are your long-term career goals?

Answer: Look for alignment with the SRE role and a commitment to growth. A strong answer might include: * Aspiring to become a senior SRE or lead a reliability engineering team. * Gaining expertise in specific technologies (e.g., Kubernetes, cloud architecture). * Contributing to open-source projects or the SRE community. * Developing skills in related areas like security, performance engineering, or data engineering. * Continuing education through certifications or advanced degrees. * Mentoring junior engineers and sharing knowledge.

This list is not exhaustive but provides a comprehensive view of what a good answer might include.

---

8. How do you handle stressful situations, such as outages or tight deadlines?

Answer: Look for strategies that demonstrate composure and effectiveness under pressure. A strong answer might include: * Staying calm and focused on the task at hand. * Following established incident response procedures. * Communicating clearly with the team and stakeholders. * Breaking down the problem into smaller, manageable parts. * Prioritizing tasks based on impact and urgency. * Taking short breaks if needed to maintain clarity.

This list is not exhaustive but provides a comprehensive view of what a good answer might include.

Round 1 mixed Questions & Detailed Answers

1. Explain what a CI/CD pipeline is and why it's critical in Site Reliability Engineering.

Answer: A CI/CD pipeline automates the software delivery process from code commit to deployment.

- **CI (Continuous Integration)** ensures that code changes are automatically built and tested, detecting issues early.
- **CD (Continuous Deployment/Delivery)** automates the release process, reducing human errors and enabling faster rollouts.

For an SRE, CI/CD pipelines are critical because:

- They improve developer velocity without compromising reliability.

- Automated testing ensures that code is validated before reaching production.
- Rollbacks and versioning help recover quickly from failures.
- Monitoring integrations help in tracking deployments and diagnosing issues faster.

---

2. How do you use Terraform in managing cloud infrastructure? Explain with an example.

Answer: Terraform is an open-source IaC tool that manages infrastructure through declarative configuration files.

Example:

```
provider "google" {
  project = "my-gcp-project"
  region  = "us-central1"
}

resource "google_compute_instance" "default" {
  name         = "my-vm"
  machine_type = "e2-medium"
  zone         = "us-central1-a"

  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-10"
    }
  }

  network_interface {
    network = "default"
    access_config {}
  }
}
```

This configuration defines a VM instance. Applying it with `terraform apply` provisions the resource. Terraform maintains state and can plan updates or deletions safely, ensuring infrastructure changes are predictable and reversible.

---

3. What is Helm, and how does it help with Kubernetes deployments?

Answer: Helm is a package manager for Kubernetes that simplifies deploying and managing applications using charts (pre-configured application templates).

Benefits:

- **Versioning**: Helm charts can be versioned, enabling easy rollbacks.
- **Reusability**: Templates allow reuse across environments.
- **Parameterization**: Configuration is abstracted and can be easily overridden for different deployments.
- **Automation**: Helm integrates with CI/CD pipelines, reducing manual configurations.

For example, deploying a web app with a Helm chart allows setting environment-specific parameters such as image version, resource limits, and secrets without changing the base configuration.

---

4. How would you approach minimizing downtime during a deployment?

Answer: Several deployment strategies help minimize downtime:

1. **Blue-Green Deployment**:
   - Maintain two identical environments.
   - Route traffic to the new environment only after successful deployment.
2. **Rolling Updates**:
   - Gradually update pods to prevent service disruption.
   - Kubernetes orchestrates health checks during rollout.
3. **Canary Releases**:
   - Route a small percentage of traffic to the new version.
   - Monitor performance before full rollout.

Additionally:

- Pre-deployment tests (integration, smoke tests).
- Automated rollback on failures.
- Monitoring with Prometheus/Grafana to detect issues early.
- Ensuring database schema changes are backward compatible.

---

5. Explain how Prometheus and Grafana help with observability in production systems.

Answer: Prometheus is a monitoring system that collects metrics (CPU usage, memory, request rates) and stores them in a time-series database. It allows querying, alerting, and analyzing trends.

Grafana is a visualization tool that reads Prometheus metrics and displays them as dashboards.

Together they:

- Provide insights into system health and performance.
- Help track incidents by correlating spikes or anomalies.
- Enable alerts for predefined thresholds (e.g., high latency).
- Support capacity planning by analyzing usage patterns over time.

For an SRE, this combination is essential for improving Mean Time to Detect (MTTD) and resolving issues faster.

---

6. What is IAM and RBAC, and why is security important in an SRE role?

Answer: IAM (Identity and Access Management) ensures that only authorized users and services can access resources.

RBAC (Role-Based Access Control) defines permissions based on roles, minimizing the attack surface by granting least privilege access.

Security is critical because:

- Misconfigurations can lead to data breaches or service interruptions.
- Access needs to be controlled, audited, and traceable.
- Automation and deployment pipelines must be secured to prevent unauthorized changes.

For example, configuring Kubernetes with RBAC policies ensures that only specific pods or users can access sensitive APIs.

---

7. What challenges have you faced with Kubernetes rollouts, and how did you resolve them?

Answer: Common challenges include:

- **Deployment failures due to misconfigured health checks** → Fixed by setting readiness and liveness probes.
- **Resource contention** → Configured appropriate CPU and memory limits.
- **Image version mismatches** → Used Helm values files to standardize deployments.
- **Manual errors** → Automated rollouts using GitOps and CI/CD integrations.
- **Monitoring gaps** → Integrated Prometheus and Grafana for proactive alerting.

Resolution involved:

- Testing pipelines thoroughly in staging.
- Automating rollback strategies.
- Enforcing peer reviews and audit trails in configuration changes.

---

8. How do you integrate automated testing into pipelines to improve reliability?

Answer:

- Add **unit tests** during the build stage to catch code errors early.
- Integrate **integration tests** after deployment to validate service interactions.
- Implement **smoke tests** before routing live traffic.
- Automate **security scans** and dependency checks.
- Fail the pipeline if critical tests fail, preventing flawed code from reaching production.

This approach ensures only validated, stable builds are deployed, reducing incidents and improving developer trust.

---

9. How do you handle outages and improve incident response?

Answer: Steps:

1. Use monitoring tools like Prometheus to detect anomalies.
2. Set up alerting thresholds to trigger incident response teams.
3. Implement runbooks with documented procedures.
4. Use tools like Slack or PagerDuty for fast communication.
5. Perform RCA (Root Cause Analysis) post-incident to prevent recurrence.
6. Integrate logs and traces for faster troubleshooting.

The goal is to reduce Mean Time to Detect (MTTD) and Mean Time to Resolve (MTTR), while ensuring incidents are well-documented for future prevention.

---

10. Explain how you manage infrastructure across multiple cloud providers like Azure and GCP.

Answer:

- Use Terraform with provider modules to manage resources declaratively across clouds.
- Standardize naming, tagging, and access policies for consistency.

- Implement shared monitoring pipelines using cloud-native tools and open-source solutions.
- Use GitOps patterns to version control infrastructure changes.
- Secure sensitive data with secrets management tools like HashiCorp Vault or cloud-native solutions.
- Adopt multi-cloud deployment strategies only when required by architecture or redundancy needs.

This approach ensures scalability, fault tolerance, and reproducibility across environments.

Jenkins Related Questions

1. What is Jenkins and how does it fit into CI/CD pipelines?

Answer: Jenkins is an open-source automation server that helps build, test, and deploy code. It integrates with various tools and services to automate workflows in the software delivery process. In a CI/CD pipeline, Jenkins orchestrates stages such as code checkout, testing, artifact creation, and deployment, ensuring faster and reliable releases.

---

2. How do you handle pipeline failures in Jenkins?

Answer: Pipeline failures are handled by:

- Analyzing logs to identify the root cause.
- Using built-in notifications (email, Slack) to alert the team.
- Adding retry strategies for transient failures.
- Integrating tests that fail gracefully.
- Implementing rollback steps in case of critical issues.

---

3. What are shared libraries in Jenkins?

Answer: Shared libraries are reusable scripts and functions stored in a central location that can be used across multiple pipelines. They help enforce best practices, reduce duplication, and standardize workflows by sharing common steps like notifications, error handling, or deployment processes.

Python Related Questions

1. How can Python be used in automation tasks within SRE?

Answer: Python is widely used to automate tasks such as:

- Infrastructure provisioning using APIs.
- Monitoring system metrics and sending alerts.
- Parsing logs and generating reports.
- Automating deployments with scripts.
- Creating health checks and recovery procedures.

Its simplicity, libraries, and readability make it ideal for scripting and automation.

---

2. What are Python's best practices for writing production-ready automation scripts?

Answer:

- Modular code with functions and classes.
- Use logging instead of print statements.
- Handle exceptions to prevent crashes.
- Validate inputs and outputs.
- Write unit tests.
- Use environment variables or config files for sensitive data.

Git Related Questions

1. How do you manage branching strategies in Git for deployment pipelines?

Answer: Common strategies include:

- **GitFlow**: Use feature branches and merge into develop or release branches.
- **Trunk-based development**: Keep small, frequent commits to the main branch with automated tests.
- **Environment branches**: Maintain branches like `staging` and `production` to reflect deployment environments.

Automation ensures that merging and testing workflows are consistent.

---

2. How do you resolve merge conflicts in Git?

Answer:

- Identify conflicting files.
- Review changes from both branches.
- Manually edit the file to combine or choose between changes.
- Test after resolving.

- Commit the resolved version with proper messages.

Linux Related Questions

1. What Linux commands are commonly used in troubleshooting server issues?

Answer:

- `top`, `htop`: Monitor processes and resource usage.
- `df -h`: Check disk space.
- `free -m`: Check memory usage.
- `journalctl`: Review logs.
- `netstat`, `ss`: Inspect network connections.
- `ps aux`: View running processes.
- `curl`, `ping`: Check network connectivity.

---

2. How do you manage permissions in Linux?

Answer:

- Use `chmod` to set file permissions.
- Use `chown` to change ownership.
- Implement users and groups with `useradd`, `groupadd`.
- Limit access with `sudo` configurations.
- Use SSH key-based authentication instead of passwords.

Automation Related Questions

1. What is the role of automation in SRE?

Answer: Automation reduces manual intervention, improves reliability, and speeds up incident response. It's used for provisioning infrastructure, scaling applications, deploying changes, monitoring systems, and executing recovery procedures. Automation ensures consistency, reduces human error, and helps in maintaining service uptime.

---

2. How would you automate a recovery process for a failed service?

Answer:

- Detect failure using monitoring tools.
- Trigger an automated restart or failover.

- Run diagnostic checks and notify teams.
- Roll back to a previous stable version if necessary.
- Use scripts or orchestration tools like Ansible or Terraform to ensure consistent recovery.

SRE Related Questions

1. What is the difference between availability and reliability?

Answer:

- **Availability** measures how often a system is operational and accessible (uptime percentage).
- **Reliability** refers to how consistently the system performs as expected over time without failures.

A system can be highly available but not reliable if it keeps failing but recovers quickly.

---

2. Explain what "Mean Time to Detect (MTTD)" and "Mean Time to Resolve (MTTR)" are.

Answer:

- **MTTD**: The average time it takes to identify an issue or failure.
- **MTTR**: The average time it takes to fix the issue and restore service.

Reducing these metrics improves incident response and system resilience.

---

3. What strategies do you use to scale systems efficiently?

Answer:

- Implement auto-scaling based on metrics like CPU usage or request rates.
- Use caching layers to reduce load on servers.
- Distribute traffic using load balancers.
- Optimize database queries and indexing.
- Partition workloads across regions or clusters.

Jenkins Questions & Answers (Detailed)

1. What is a Jenkins pipeline and how is it defined?

Answer: A Jenkins pipeline is a set of automated steps that allow building, testing, and deploying software. Pipelines are defined using a DSL called *Pipeline as Code*, typically written in a `Jenkinsfile`. It defines stages (build, test, deploy), steps, environment variables, and error handling, allowing repeatable and version-controlled workflows.

Example Problem: Write a simple declarative pipeline that builds a project and runs unit tests.

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building the project...'
                sh 'make build'
            }
        }
        stage('Test') {
            steps {
                echo 'Running tests...'
                sh 'make test'
            }
        }
    }
}
```

---

2. How do you trigger Jenkins jobs automatically?

Answer: Jobs can be triggered in several ways:

- Polling a Git repository periodically.
- Using webhooks from version control systems.
- Scheduling builds via `cron` syntax.
- Triggering downstream jobs after completion of others.

This helps integrate Jenkins into a fully automated CI/CD pipeline.

Example Problem: Explain how you would configure a job to poll a Git repository every 15 minutes.

Answer: In the job's configuration under *Build Triggers*, select "Poll SCM" and enter the schedule as:

```
H/15 * * * *
```

This will check for new commits every 15 minutes and trigger builds when changes are detected.

---

3. What is the purpose of Jenkins shared libraries?

Answer: Shared libraries enable reusing code (functions, scripts, common steps) across multiple pipelines. This reduces duplication, enforces standards, and improves maintainability. It's especially useful when several projects share similar build or deployment logic.

Example Problem: Describe how you would structure a shared library to manage deployment steps.

Answer:

- Create a `vars/` directory with reusable scripts like `deploy.groovy`.
- Define parameters for environment, version, etc.
- Import the shared library in the pipeline using `@Library('common-lib') _`.

---

4. How do you handle sensitive information like API keys in Jenkins pipelines?

Answer: Sensitive data should be handled securely using:

- Jenkins credentials manager.
- Environment variables scoped to jobs.
- Vault or external secret management systems.

Example: Using credentials plugin to inject API tokens during runtime without hardcoding them in the pipeline.

---

5. How can you rollback a deployment if something goes wrong?

Answer: Implement rollback using:

- Version control: revert to a previous stable build.
- Helm rollback for Kubernetes deployments.
- Using Jenkins pipeline steps to trigger rollback commands upon failure.

You can also monitor health after deployment and trigger rollback based on alerts.

---

6. Explain how you monitor Jenkins pipelines for failures.

Answer: Monitoring involves:

- Notifications via email, Slack, or webhooks.
- Integration with Prometheus exporters.
- Using pipeline logs to trace failed stages.
- Alerts on error thresholds or slow builds.

Regular monitoring helps reduce downtime and improve deployment success.

---

7. Write a Jenkinsfile step to checkout code from a Git repository.

```
pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps {
                git url: 'https://github.com/example/repo.git', branch: 'main'
            }
        }
    }
}
```

Git Questions & Answers (Detailed)

1. Explain how branching works in Git and why it's useful.

Answer: Branching allows you to create isolated environments to develop new features or fix bugs without affecting the main codebase. Each branch has its own commit history and changes can be merged later into the main branch.

---

2. How do you resolve conflicts in Git?

Answer: Conflicts occur when two branches modify the same part of a file. To resolve them:

1. Identify the conflicting files.
2. Edit the file to combine or choose between changes.
3. Mark the conflict as resolved using `git add`.
4. Commit the resolved version.

---

3. What is the difference between `git merge` and `git rebase`?

Answer:

- `git merge` creates a new commit combining histories of two branches.
- `git rebase` rewrites commit history by applying changes on top of another branch.

Rebase creates a cleaner, linear history while merge retains all intermediate commits.

---

4. How can you undo a commit in Git?

Answer: Options include:

- `git reset --soft HEAD~1`: Undo commit but keep changes staged.
- `git reset --hard HEAD~1`: Undo commit and discard changes.
- `git revert <commit>`: Create a new commit that undoes the changes.

---

5. Write a command to clone a repository and checkout a specific branch.

```
git clone https://github.com/example/repo.git
cd repo
git checkout feature-branch
```

---

6. How do you tag a release in Git?

Answer: Use the following commands to tag and push:

```
git tag -a v1.0 -m "First stable release"
git push origin v1.0
```

Tags help mark versions or releases for deployment.

---

7. How do you view commit history in Git?

```
git log --oneline --graph --decorate
```

This shows a simplified commit history with branching structure.

---

8. How do you stash changes temporarily?

```
git stash
```

This saves your work in progress and cleans the working directory for tasks like switching branches.

Linux Questions & Answers (Detailed)

1. How do you check resource usage on a Linux server?

Answer: Commands like:

- `top` or `htop`: Real-time resource monitoring.
- `df -h`: Disk space usage.
- `free -m`: Memory usage.
- `iostat`: Disk and CPU I/O stats.
- `vmstat`: System performance overview.

---

2. How do you check which processes are consuming high memory?

```
ps aux --sort=-%mem | head -n 10
```

This shows the top 10 processes using the most memory.

---

3. How do you set file permissions?

```
chmod 644 file.txt
```

This grants read and write permissions to the owner and read-only permissions to others.

---

4. How do you find a specific pattern in logs?

```
grep "ERROR" /var/log/syslog
```

This filters all lines containing "ERROR".

---

5. How do you monitor disk I/O in real time?

```
iostat -xz 2
```

This shows detailed input/output statistics every 2 seconds.

---

6. How do you kill a process by its PID?

```
kill -9 <PID>
```

The `-9` flag forces the process to terminate.

---

7. How do you search for a package and install it using apt?

```
sudo apt search nginx
sudo apt install nginx
```

This helps automate package management.

---

8. How do you set environment variables temporarily and permanently?

Answer:

- Temporary: `export VAR=value` (only lasts for the session).
- Permanent: Add `export VAR=value` to `~/.bashrc` or `~/.profile`.

Automation Questions & Answers (Detailed)

1. What is infrastructure as code (IaC), and why is it important?

Answer: IaC defines infrastructure using code, enabling automation, version control, and reproducibility. It helps create, update, and manage environments consistently, reducing errors and speeding up deployments.

---

2. How do you ensure scripts are idempotent?

Answer: Idempotent scripts produce the same outcome even when run multiple times by:

- Checking if resources exist before creating them.
- Using flags that update existing resources.
- Avoiding destructive operations without confirmation.

---

3. Write a Python script to automate backup by copying files.

```
import shutil
import datetime

source = '/path/to/source'
destination = f'/path/to/backup/backup_{datetime.datetime.now().strftime("%Y%m%d%H%M%S")}'

shutil.copytree(source, destination)
print(f'Backup completed: {destination}')
```

---

4. How do you schedule tasks with cron?

Answer: Add entries in the crontab using `crontab -e`. For example, to run a script every day at midnight:

```
0 0 * * * /usr/bin/python3 /path/to/script.py
```

---

5. How do you use APIs in automation scripts?

Answer: Use the `requests` library to send HTTP requests, handle authentication, and parse JSON responses.

Example Problem:

```
import requests

response = requests.get("https://api.example.com/status")
if response.status_code == 200:
    data = response.json()
    print("Status:", data["status"])
else:
    print("Failed to fetch status")
```

---

6. How do you manage secrets in automation?

Answer:

- Store in environment variables.
- Use encrypted files or vaults.
- Avoid hardcoding credentials.
- Use secret management tools like HashiCorp Vault or AWS Secrets Manager.

---

7. Write a Bash script to monitor if a service is running.

```
#!/bin/bash

if systemctl is-active --quiet nginx; then
    echo "nginx is running"
else
    echo "nginx is not running"
    systemctl start nginx
fi
```

---

8. How do you test automation scripts before deploying them?

Answer:

- Run in a staging environment.
- Include unit and integration tests.
- Use mock data and dry runs.
- Validate output against expected results.

SRE Questions & Answers (Detailed)

1. What is the role of observability in reliability?

Answer: Observability allows teams to understand the internal state of systems through metrics, logs, and traces. It helps detect anomalies, understand failures, and proactively improve system uptime.

---

2. How would you design alerts without causing alert fatigue?

Answer:

- Use severity levels (critical vs warning).
- Aggregate related alerts.
- Tune thresholds based on historical data.
- Suppress alerts during maintenance windows.
- Provide actionable insights in alert messages.

---

3. How do you implement fault tolerance?

Answer:

- Use redundant components.

- Implement retries with exponential backoff.
- Design failover strategies.
- Monitor health checks and trigger alerts.
- Use load balancing and geographic distribution.

---

4. Write pseudo-code for an incident response checklist.

```
1. Acknowledge alert
2. Identify affected service
3. Check monitoring dashboards
4. Inform stakeholders
5. Run diagnostics
6. Apply quick fixes if safe
7. Rollback changes if needed
8. Document the issue and fix
9. Perform RCA post-resolution
```

---

5. Explain how you measure system uptime.

Answer: System uptime is calculated as the percentage of time the system remains operational over a period.

Formula:

```
Uptime % = (Total time - Downtime) / Total time × 100
```

Tools like Prometheus and monitoring services track this metric continuously.

---

6. How do you balance performance and reliability?

Answer:

- Optimize caching while ensuring stale data is handled.
- Scale resources only when needed.
- Use rate limiting to prevent overload.
- Implement fallback mechanisms in case of failures.

---

7. How do you test disaster recovery?

Answer:

- Simulate failures in staging or test environments.
- Validate backup restoration.

- Test failover between regions or clusters.
- Automate recovery procedures and ensure rollback mechanisms.

---

8. What strategies help reduce manual intervention in operations?

Answer:

- Automate common tasks with scripts.
- Use CI/CD pipelines for deployments.
- Integrate alerting and incident management.
- Document runbooks for troubleshooting.
- Implement monitoring dashboards for real-time insight.

Python Questions & Answers

1. What are Python's data types, and how do they help in automation scripts?

Answer: Python supports several built-in data types that help store and manipulate data efficiently in automation tasks:

- **int, float**: Handle numeric values like CPU usage or memory statistics.
- **str**: Work with log messages or configuration files.
- **list, tuple**: Store sequences of items, like servers or IP addresses.
- **dict**: Store key-value pairs, like metadata or API responses.
- **set**: Store unique items for efficient lookup.
- **bool**: Represent binary states like service up/down.

Using appropriate data types ensures the script processes data efficiently and prevents runtime errors.

Example Problem: Write a script that reads a list of IP addresses and removes duplicates before returning the sorted list.

```
def process_ips(ip_list):
    unique_ips = sorted(set(ip_list))
    return unique_ips

# Example
ips = ["192.168.1.1", "10.0.0.1", "192.168.1.1"]
print(process_ips(ips))  # Output: ['10.0.0.1', '192.168.1.1']
```

---

2. How do you handle exceptions in Python to ensure scripts don't crash unexpectedly?

Answer: In automation scripts, unexpected errors like missing files or network issues can crash the program. Python's `try-except` blocks allow handling errors gracefully:

- `try`: The block of code to attempt.
- `except`: Handles specific or general exceptions.
- `finally`: Executes cleanup actions regardless of success or failure.

This helps ensure scripts continue running or fail with clear messages.

Example Problem: Write a function that opens a file and reads its contents. Handle the case where the file doesn't exist.

```
def read_file(file_path):
    try:
        with open(file_path, 'r') as f:
            return f.read()
    except FileNotFoundError:
        return "File not found."
    except Exception as e:
        return f"An error occurred: {e}"

# Example
print(read_file("nonexistent.txt"))
```

---

3. Explain how list comprehensions help in writing concise automation scripts.

Answer: List comprehensions are a Pythonic way to create lists using a single line of code. They make scripts shorter and more readable by combining loops and conditionals.

Example Problem: Given a list of CPU usage percentages, create a list of those that exceed 80%.

```
cpu_usages = [45, 82, 67, 90, 77]
high_usage = [usage for usage in cpu_usages if usage > 80]
print(high_usage)  # Output: [82, 90]
```

This helps in monitoring systems and filtering metrics quickly.

---

4. How do you work with dictionaries in Python for configuration or monitoring data?

Answer: Dictionaries are ideal for storing structured information like system metrics, API responses, or configuration parameters. You can easily add, update, and access data.

Example Problem: Write a script that counts occurrences of error messages in a list.

```
def count_errors(errors):
    error_count = {}
    for err in errors:
        error_count[err] = error_count.get(err, 0) + 1
    return error_count

errors = ["404", "500", "404", "502", "500", "404"]
print(count_errors(errors))
# Output: {'404': 3, '500': 2, '502': 1}
```

This is helpful in log aggregation or troubleshooting.

---

5. What are Python's built-in modules you use for automation and monitoring?

Answer: Common modules include:

- **os**: Interact with the operating system, manage files and processes.
- **sys**: Access system-specific parameters and arguments.
- **subprocess**: Run shell commands from Python scripts.
- **requests**: Send HTTP requests to APIs.
- **time**: Pause execution, timestamp operations.
- **logging**: Generate structured logs for debugging.
- **json**: Parse or create JSON data for APIs or configuration.

These modules help build robust automation scripts that interact with infrastructure and external systems.

Example Problem: Write a script that runs a shell command and prints its output.

```
import subprocess

def run_command(cmd):
    result = subprocess.run(cmd, shell=True, capture_output=True, text=True)
    if result.returncode == 0:
        return result.stdout.strip()
    else:
```

```
        return f"Error: {result.stderr.strip()}"

print(run_command("echo 'Hello from Python!'"))
```

---

6. How do you implement logging in Python scripts for debugging?

Answer: The `logging` module is used to create logs at different levels (DEBUG, INFO, WARNING, ERROR). It's better than using `print()` because logs can be filtered, formatted, and saved.

Example Problem: Create a simple logger that logs messages to the console.

```
import logging

logging.basicConfig(level=logging.DEBUG, format='%(asctime)s - %(levelname)s - %(message)s')

logging.debug("This is a debug message")
logging.info("Monitoring started")
logging.warning("CPU usage high")
logging.error("Service not responding")
```

This helps track issues without cluttering the console.

---

7. How do you schedule tasks in Python without external tools?

Answer: You can use the `time` module's `sleep()` function inside loops to periodically run tasks. For more complex scheduling, you can use libraries like `schedule` or `APScheduler`.

Example Problem: Write a script that prints "Health check running..." every 5 seconds, 3 times.

```
import time

for i in range(3):
    print("Health check running...")
    time.sleep(5)
```

This is useful for writing lightweight monitoring scripts.

---

8. Write a Python script to parse JSON data and extract important information.

Answer: In SRE, APIs or configuration files often return JSON data that needs to be parsed and analyzed.

Example Problem: Given a JSON string representing system metrics, extract the CPU usage.

```
import json

data = '{"system": "web-server", "metrics": {"cpu": 85, "memory": 70}}'
parsed = json.loads(data)

cpu_usage = parsed["metrics"]["cpu"]
print(f"CPU Usage: {cpu_usage}%")
```

This helps automate system health reporting.

Ending Questions & Answers

1. Do you have any questions for us?

---

2. Is there anything else you would like to share about your experience?

---

3. How do you see yourself contributing to our team?

---

4. Why should we hire you for this role?

---

5. What are your expectations from this role and the company?
